



NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

STR

LAB #1 – class 5/5

freeRTOS: TASKS & SYNCHRONIZATION,
COMMUNICATION, INTERRUPTS & TASK CONTROL

LABWORK 1 ARCHITECTURE TIPS

2025 - 2026

freeRTOS: Tasks and Synchronization with Semaphores & communication with mailboxes

- ☐ Task Creation Review
- ☐ Race condition Review
- ☐ Semaphores Review
- ☐ Mailboxes
- ☐ Labwork 1: First tasks identification and interactions (synchronization & communication)
- ☐ Interrupts Review
- ☐ Task Control (task suspend & task resume)

Lab Work 1

REVIEW

NOVA

Let's architect our project solution!



NOVA NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

Departamento de Engenharia Electrotécnica e de Computadores
Secção de Robótica e Manufatura Integrada

Description of Lab Work nº 1	
Course	Real-Time systems / Sistemas de Tempo Real
Year	2025/2026
Aim	Study concepts of Real-Time Systems and develop a project with a real-time kernel
Classes	5+1 classes x 3 h
Delivery Date	27/10/2023

Concrete Objectives:

1. Interaction with physical systems:
 - a. Interface board (Data acquisition)
 - b. Sensors
 - c. Actuators
 - d. Interface functions (in C++)
2. Apply the base real-time system/
 - a. Task / Process: concurrent
 - b. State, event, actions (triggers)
 - c. Synchronization and communication
 - d. Accessing shared resources
3. Using Real-Time kernels
 - a. freeRTOS
 - b. C/C++ Language
4. Solve the problem described in A

NOVA NOVA SCHOOL OF
SCIENCE & TECHNOLOGY

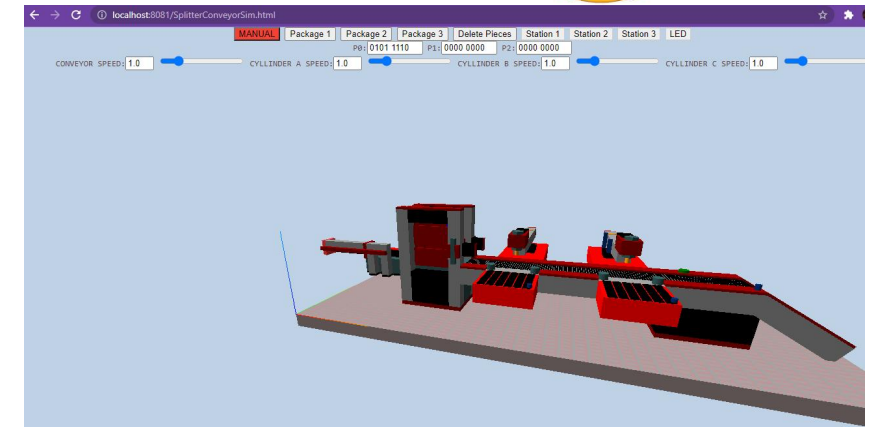
Departamento de Engenharia Electrotécnica e de Computadores
Secção de Robótica e Manufatura Integrada

Finally, keep in mind that the system has active high and active low sensors. Active low sensors provide logical value false/0 when active, and true/1 otherwise.

The functional **requirements** to develop the **Splitter Lego Conveyor** are listed in the following table. They are split into basic or low-level requirements, for direct interaction with sensors and actuators, and high-level requirements, for storage management.

Table 1 – Needed requirements.

Req.	Description
Low-Level Requirements:	
FR1.1	Move and stop each cylinder
FR1.2	Move and stop moving the conveyor
FR1.3	Read Brick identification sensors
FR1.4	Detect the button/switches states
FR1.5	Turn Lamp on and off
FR1.6	Keyboard or UI interaction
High-Level requirements (operation):	
FR2	Using the keyboard, develop robust (semi-automatic) calibration . Using keys, the user defines the direction of the movements. During a movement, the actuator must stop when it reaches a position sensor (Cylinders)
FR3	Insert a brick into the system, which gets a brick from the entry queue and puts it in the conveyor
FR4	By pressing "s" the system Starts its working cycle. Pressing "f", the system Finishes its work.
FR5	Deliver Brick1 for Dock 1 , Brick2 for Dock 2 , and Brick3 to DockEnd
FR6	Detection of buttons P1.4 and P1.3 pressed by human operator, at dock 1 and dock 2 to force specific bricks to the corresponding docks
FR7	Flash lamp at P2.7 once when delivering sequence at Dock 1 or Dock 2



➤ Need a solution to perform the required functionalities!

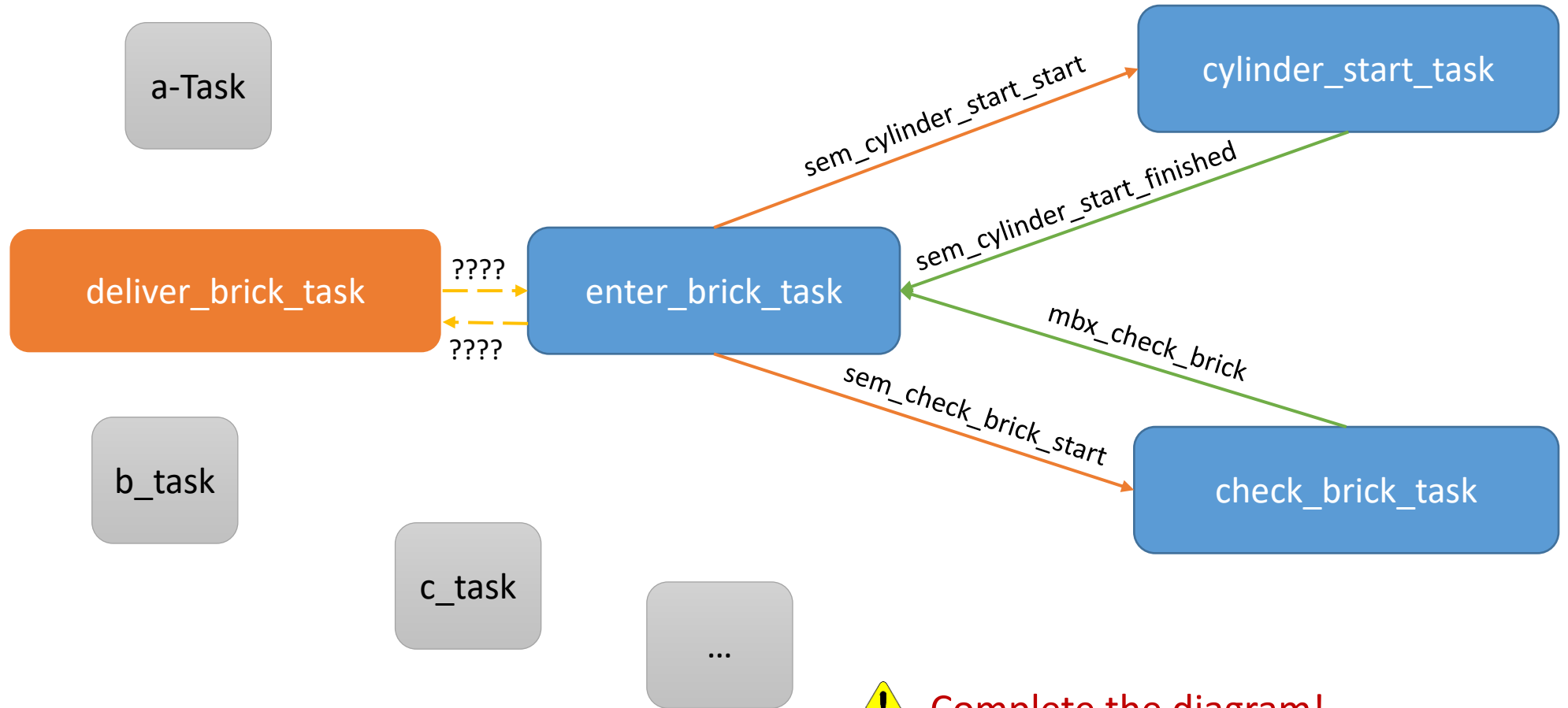
- Menu showing functionalities
- Request user to insert brick
- Performing identification task
- Sending brick to correct dock
- Etc...

❖ Which tasks?

❖ Need tasks synchronization?

❖ Need tasks communication? (information exchange)

□ Let's start by designing a diagram of tasks and inter-task interactions:



Complete the diagram!

□ Definition and creation of semaphores and mailboxes

```
//Semaphores and Mailboxes declaration
xSemaphoreHandle sem_cylinder_start_start;
xSemaphoreHandle sem_check_brick_start;
xSemaphoreHandle sem_cylinder_start_finished;

xQueueHandle mbx_check_brick;
```

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

    sem_cylinder_start_start = xSemaphoreCreateCounting(10, 0);
    sem_check_brick_start = xSemaphoreCreateCounting(10, 0);
    sem_cylinder_start_finished = xSemaphoreCreateCounting(10, 0);

    mbx_check_brick = xQueueCreate(10, sizeof(int));
}
```

❑ Definition and creation of cylinder_start_task, check_brick_task and enter_brick_task

```
void cylinder_start_task (void* pvParameters) {  
    while (TRUE) {  
        xQueueSemaphoreTake(sem_cylinder_start_start,  
portMAX_DELAY);  
        gotoCylinderStart(1);  
        xSemaphoreGive(sem_cylinder_start_finished);  
        gotoCylinderStart(0); // signals task completion  
    }  
}
```

```
void check_brick_task(void* pvParameters) {  
    int brickType;  
    while (TRUE) {  
        xQueueSemaphoreTake(sem_check_brick_start,  
portMAX_DELAY);  
        brickType=1; // to be replaced by check brick function later  
        xQueueSend(mbx_check_brick, &packageType,  
portMAX_DELAY); // task completion  
    }  
}
```

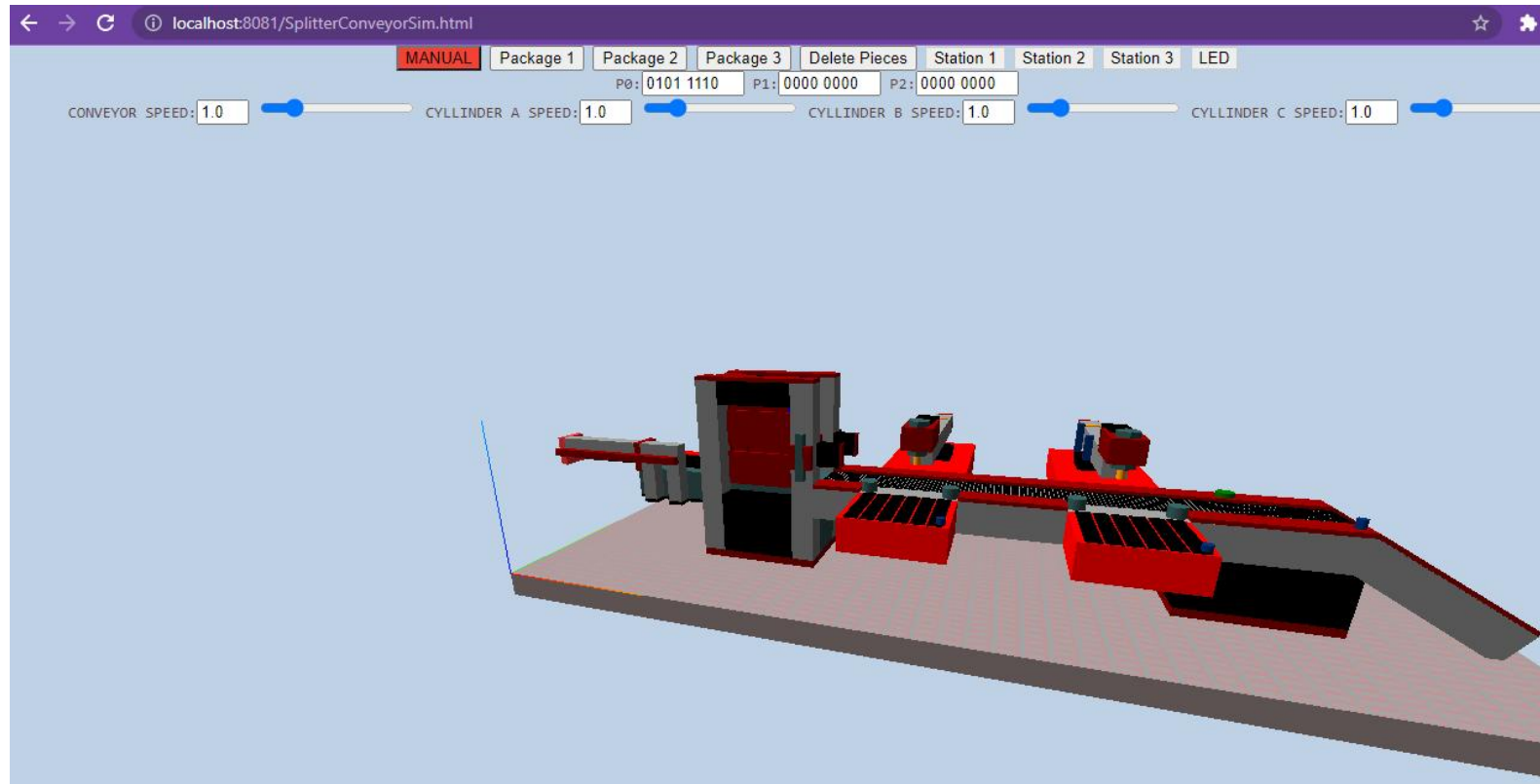
send **request** for cylinder_start_task and check_brick_task which execute gotoCylinderStart and checkBrick on their own.

```
void enter_brick_task(void* pvParameters) {  
    int brickType;  
    while (TRUE) {  
        _getch(); // press a key to enter a new package  
        xSemaphoreGive(sem_cylinder_start_start);  
        xSemaphoreGive(sem_check_brick_start);  
  
        xQueueSemaphoreTake(sem_cylinder_start_finished, portMAX_DELAY);  
        xQueueReceive(mbx_check_brick, &brickType, portMAX_DELAY);  
  
        printf("Brick received, Brick Type: %d", brickType);  
    }  
}
```

// wait for tasks completion

```
void myDaemonTaskStartupHook(void) {  
    inicializarPortos();  
    sem_cylinder_start_start = xSemaphoreCreateCounting(10, 0);  
    sem_check_brick_start = xSemaphoreCreateCounting(10, 0);  
    sem_cylinder_start_finished = xSemaphoreCreateCounting(10, 0);  
    mbx_check_brick = xQueueCreate(10, sizeof(int));  
    xTaskCreate(cylinder_start_task, "Cylinder Start Task", 100, NULL, 0, NULL);  
    xTaskCreate(check_brick_task, "Check brick Task", 100, NULL, 0, NULL);  
    xTaskCreate(enter_brick_task, "Enter brick Task", 100, NULL, 0, NULL);  
}
```

□ Run the project...



INTERRUPTS [REVIEW]

REVIEW

NOVA

□ Using Interrupt Service Routines (ISR) on port values

freeRTOS/include -> interrupt.h:

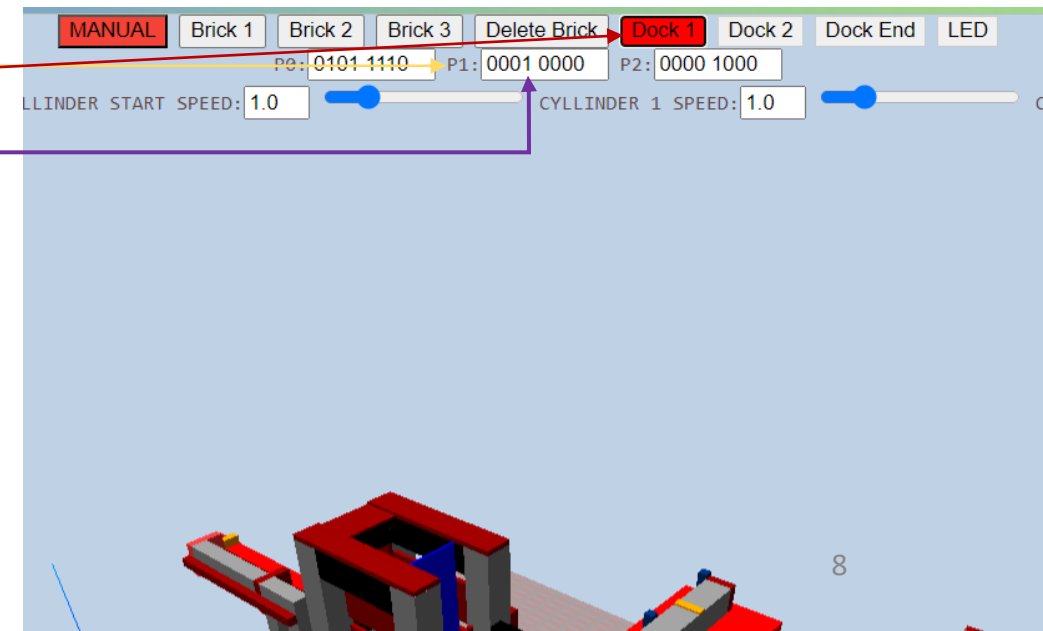
```
void attachInterrupt(int port, int bit, void(*ISR)(ULONGLONG lastTime), int Mode);
```

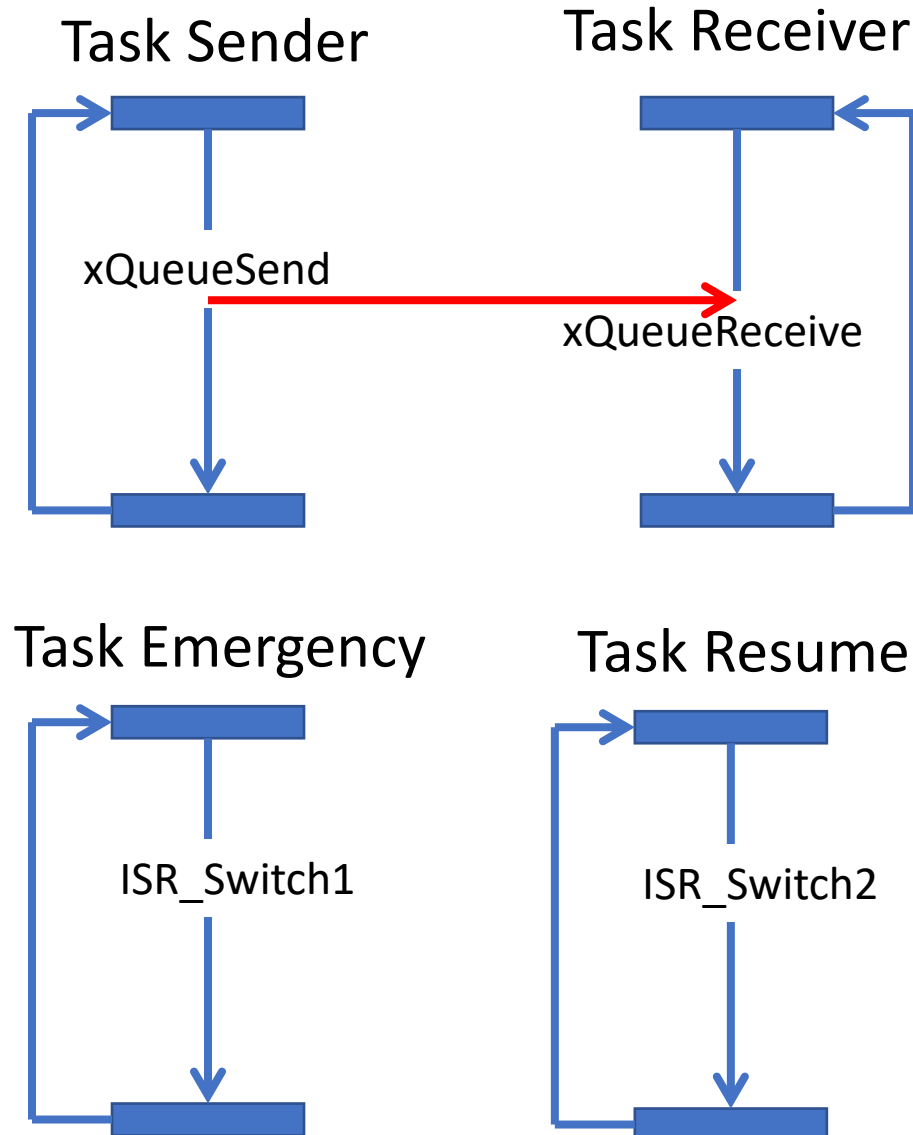
Example:

switch1 -> p1.4 (active high)

```
void myDaemonTaskStartupHook(void) {  
    // ...code already here  
    attachInterrupt(1, 4, switch1_rising_isr, RISING);  
    attachInterrupt(1, 4, switch1_falling_isr, FALLING);  
    attachInterrupt(1, 3, switch2_change_isr, CHANGE);  
}
```

Mode	LOW	Triggers interrupt whenever the pin is LOW
	HIGH	Triggers interrupt whenever the pin is HIGH
	FALLING	Triggers interrupt when the pin goes from HIGH to LOW
	RISING	Triggers interrupt when the pin goes from LOW to HIGH
	CHANGE	Triggers interrupt whenever the pin changes value, from HIGH to LOW or LOW to HIGH





- ❑ Two parallel tasks exchanging data through a **MAILBOX**
- ❑ One emergency task triggered by an interrupt -> switch (suspends task sender)
- ❑ One resume task triggered by an interrupt -> switch (resumes task sender)

□ Let's update project `demo_rtos.sln` (in `C:\str\demo_rtos`) to try this example:

```
// tasks handler declaration
xTaskHandle emergencyTask;
xTaskHandle resumeTask;
xTaskHandle taskSender;
```

Emergency and Resume Tasks Creation & Implementation

```
void vTaskEmergency(void* pvParameters){
    // The task being suspended and resumed.
    for (;;) {
        // The task suspends itself.
        vTaskSuspend(NULL);
        // The task is now suspended, so will
        // not reach here until the ISR resumes it.
        printf("\n **** EMERGENCY task\n");
        // The task suspends task sender.
        vTaskSuspend(taskSender);
    }
}
```

```
void vTaskResume(void* pvParameters){
    // The task being suspended and resumed.
    for (;;) {
        // The task suspends itself.
        vTaskSuspend(NULL);
        // The task is now suspended, so will
        // not reach here until the ISR resumes it.
        printf("\n **** RESUME task\n");
        // The task suspends task sender.
        vTaskResume(taskSender);
    }
}
```

The handler of the
taskSender needs
to be known!

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();
    // *** some code here
    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);
}
```

The handler of the
task will be
needed by the ISR!

ISR Routines Implementation

```
void switch1_rising_isr(ULONGLONG lastTime) {
    // GetTickCount64() current time in milliseconds
    // since the system has started...
    ULONGLONG time = GetTickCount64();
    printf("\nSwitch one RISING detected at time = %llu...\n", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(emergencyTask);
}
```

```
void switch2_rising_isr(ULONGLONG lastTime) {
    ULONGLONG time = GetTickCount64();
    printf("\nSwitch two RISING detected at time = %llu...", time);

    BaseType_t xYieldRequired;
    // Resume the suspended task.
    xYieldRequired = xTaskResumeFromISR(resumeTask);
}
```

In myDaemonTaskStartupHook...

```
void myDaemonTaskStartupHook(void) {
    inicializarPortos();

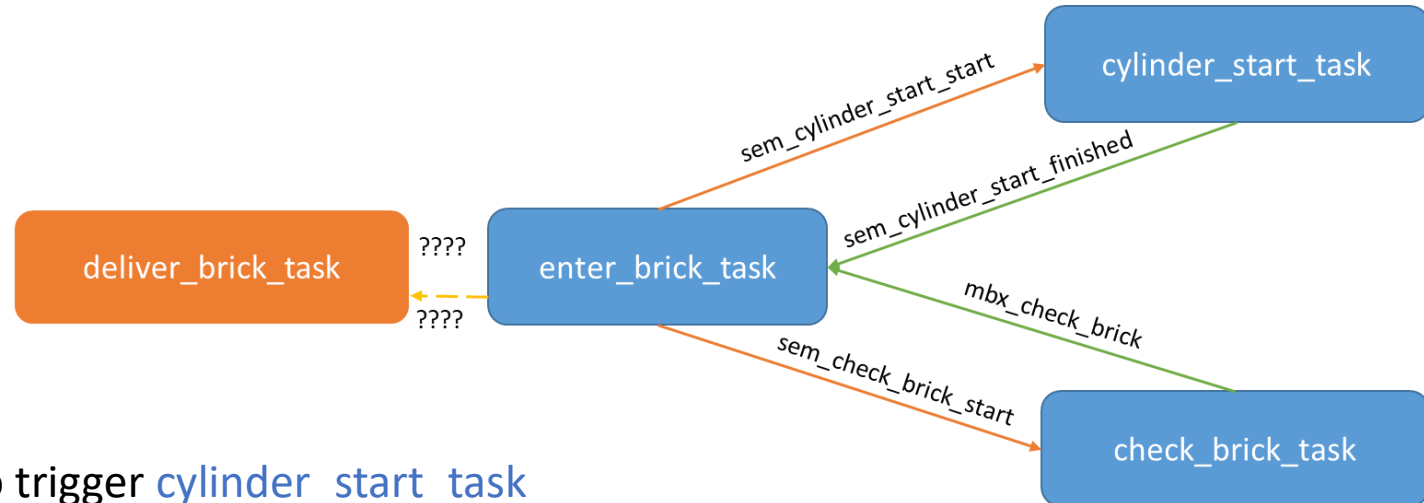
    // *** some code here

    xQueueHandle xQueue = xQueueCreate(10, sizeof(double));
    xTaskCreate(vTaskCode_MbxReceiver, "vTaskCode_MbxReceiver ", 100, xQueue, 0, &taskReceiver);
    xTaskCreate(vTaskCode_MbxSender, "vTaskCode_MbxSender ", 100, xQueue, 0, &taskSender);

    xTaskCreate(vTaskEmergency, "vTaskEmergency ", 100, NULL, 0, &emergencyTask);
    xTaskCreate(vTaskResume, "vTaskResume ", 100, NULL, 0, &resumeTask);

    attachInterrupt(1, 4, switch1_rising_isr, RISING);
    attachInterrupt(1, 3, switch2_rising_isr, RISING);
}
```

- Implementing the designed tasks and interactions (the blue ones so far...)

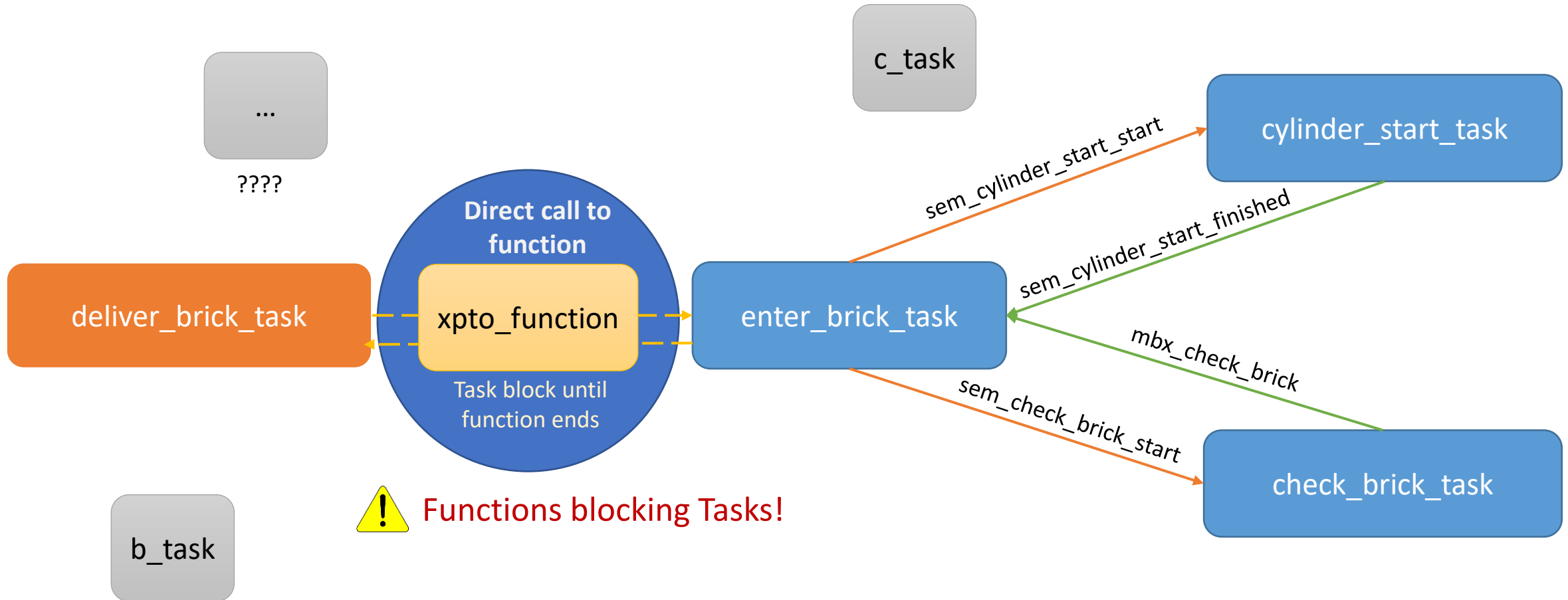


Some suggestions....

- ✓ Creation of semaphore **sem_cylinder_start_start** to trigger **cylinder_start_task**
- ✓ Creation of semaphore **sem_check_brick_start** to trigger **check_brick_task**
- ✓ Creation of semaphore **sem_cylinder_start_finished** to trigger **enter_brick_task**
- ✓ Creation of mailbox **mbx_check_brick** to communicate with **enter_brick_task**
- ✓ Creation of **enter_brick_task**
- ✓ Creation of **cylinder_start_task**
- ✓ Creation of **check_brick_task**
- ✓ Check interrupts (example P1.4)

Lab Work 1 – Architecture Tips

1. Calling a function from task deliver brick task...

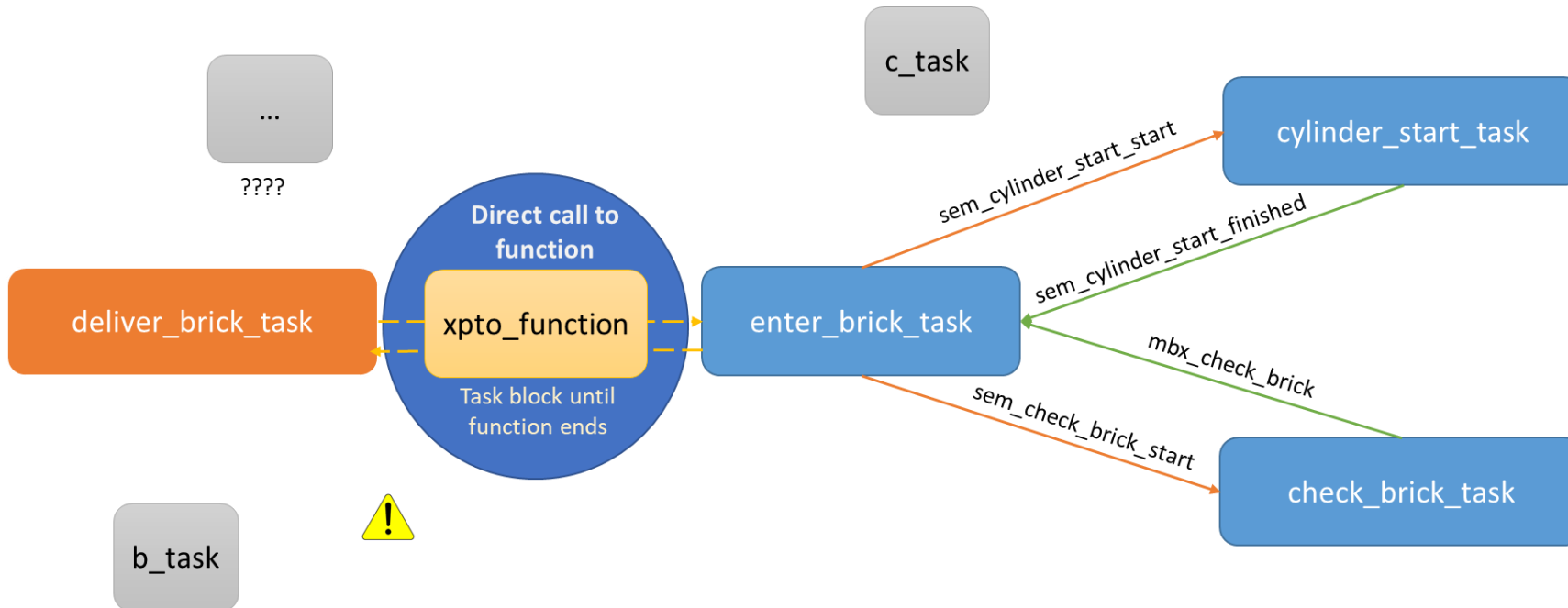


! Functions blocking Tasks!

If more tasks are being performed at the same time.... Called from `deliver_brick_task`..... They will have to wait that `xpto_function` finishes!

1. Calling a function from task deliver brick task...

- ✓ Calling a function has the effect of blocking **deliver_brick_task** until the function finishes its work. During this state, **deliver_brick_task** is not able to receive new requests and therefore can't work concurrently, react and "invoke" other tasks.



How to solve it?

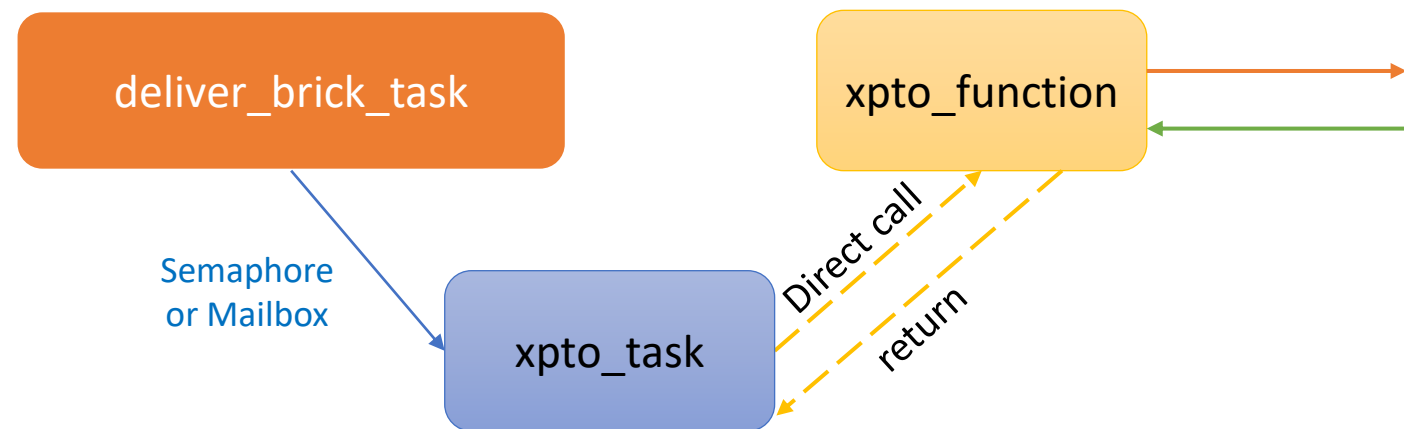
Lab Work 1 – Architecture Tips

1. Calling a function from task deliver brick task...

❑ Invoke the xpto_function from a **new task**!

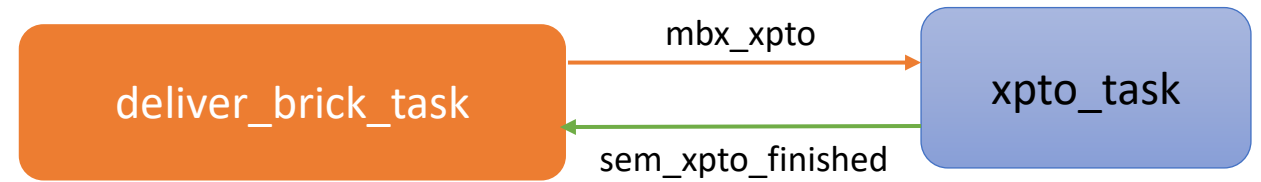


- Create an intermediate task that takes the role of waiting for a request to call **xpto_function**.
- Use synchronization or messages, as appropriate, to notify **xpto_task**.
- This way, as opposite to the direct call of **xpto_function**, deliver_brick_task can invoke it indirectly, without becoming blocked.



2. Task blocking another task...

- In this case, the **deliver_brick_task** sends a message requesting a service and then waits for **sem_xpto_finished**.
- The problem illustrated in the picture, is that **deliver_brick_task** is still blocked by semaphore **sem_xpto_finished** until the **xpto_task** is finished, and so, preventing/impeding the request of other services.



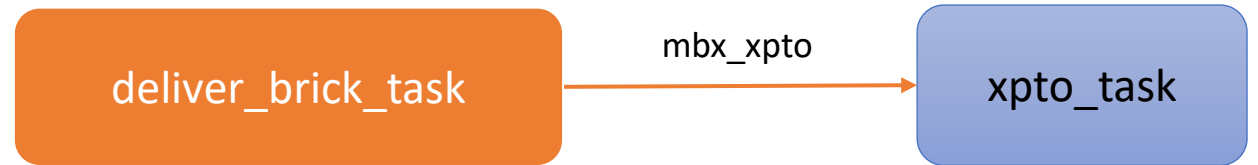
How to solve it?

2. Task blocking another task...

a) Consider removing the semaphore!



- First check whether you really need the semaphore `sem_xpto_finished`.
- If it is not necessary, then the task **deliver_brick_task** just sends the message instantaneously and proceeds to be available for other requests.

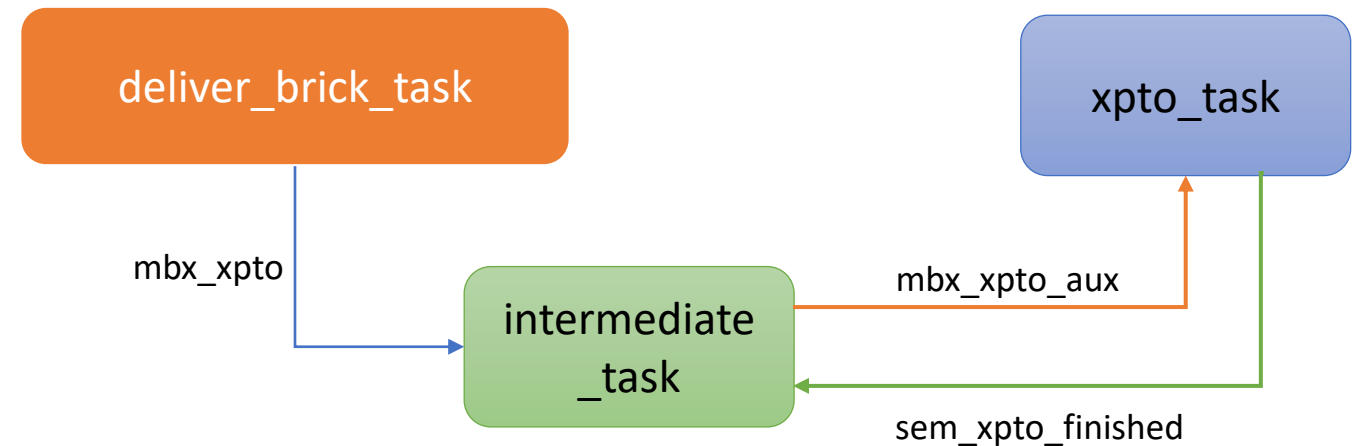


2. Task blocking another task...

b) Use an **intermediate task**!



- If a semaphore is indeed needed, the solution is to use a mediation (intermediate) task, which is assigned with the role of effectively invoking the service implementation and then waiting for its termination.
- In this way, **deliver_brick_task** does not block waiting for the semaphore, therefore, being available to receive more requests or/and user interaction.



Lab Work 1 - Suggestions

- Review Labwork 1 Requirements
- Design a Tasks and Interaction Diagram
- Brainstorm Ideas and Solutions
- Apply concepts to Labwork 1



Lab Work 1 - Remember



- Work delivery **only** through Moodle
- Upload a zip/rar archive with your Visual Studio Project.
- Each team must fill the **Report form** (*questionnaire*), which is necessary for the project evaluation. In the questionnaire it is **mandatory** to:
 - Upload a **diagram** illustrating the main tasks/functions developed in your project and their main interactions (semaphores, mailboxes, etc).
 - Upload a **video** demonstrating the main characteristics of your work (+- 5mins).

➤ **Deadline:** 27 Oct 2025 (23h59)



**SPLENDID! YOU ARE ALMOST FINISHING YOUR LAB 1 SYSTEM IN
REAL-TIME SYSTEMS!**



RELAX, EXPLORE, HAVE FUN, AND ENJOY THE RIDE!



KEEP UP THE
GOOD
WORK!